

MNIST Subset:

Binary Classification & Clustering Analysis

Digits 1 and 7 | Individual Written & Coding Assignment

Classifiers: Logistic Regression | RBF Kernel SVM

Clustering: K-Means | Optimal $k = 9$

Author - Md Soad Solaiman

Table of Contents

Table of Contents	2
Table of Figures	3
List of Tables	3
Task 1: Data Exploration and Analysis.....	4
1.1 Data Inspection	4
1.2 Class Distribution.....	4
1.3 Pixel Statistics.....	4
1.4 Average Digit Images	5
1.5 PCA Projection	5
1.6 Data Preprocessing.....	6
Task 2: Build a Classification System	7
2.1 Model Choice: Logistic Regression.....	7
2.2 Evaluation Metrics	7
2.3 Implementation & Training	7
2.4 Results.....	8
Task 3: Comparison of Two Classifiers & Hyperparameter Tuning	9
3.1 Second Classifier: RBF Kernel SVM	9
3.2 Cross-Validation Strategy	9
3.3 Logistic Regression Hyperparameter Tuning	9
3.4 RBF SVM Hyperparameter Tuning.....	10
3.5 Confusion Matrices on Training Data.....	11
3.6 Final Model Comparison	11
Task 4: Clustering Analysis of the Test Set.....	12
4.1 Preparing the Digit 7 Test Data.....	12
4.2 K-Means with 100 Clusters.....	12
4.3 Finding the Optimal Number of Clusters.....	12
4.3.1 Elbow Method (Inertia).....	12
4.3.2 Silhouette Score	13
4.4 Final Cluster Selection: $k = 9$	14
4.5.2 Cluster Centers.....	14
4.5.3 Cluster Size Distribution.....	15
4.6 Summary	15
References.....	16

Table of Figures

Figure 1: Average pixel images for digit 1 (left) and digit 7 (right)	5
Figure 2: PCA projection of training data coloured by digit class.....	5
Figure 3: Confusion matrix for Logistic Regression on the training set.....	8
Figure 4: RBF SVM hyperparameter heatmap of mean CV F1-score (C vs gamma)	10
Figure 5: Confusion matrix for RBF Kernel SVM on the training set.....	11
Figure 6: Elbow method: Inertia vs number of clusters (k=2 to 99).....	13
Figure 7: Silhouette scores for k=2 to 99.....	13
Figure 8: K-Means cluster centers (k=9) for digit 7 test images.....	14
Figure 9: Distribution of samples per cluster for digit 7 (k=9)	15

List of Tables

Table 1: Dataset component shapes	4
Table 2: Class distribution in training set	4
Table 3: Pixel statistics per digit class	4
Table 4: Preprocessing steps and justifications.....	6
Table 5: Evaluation metrics and justifications.....	7
Table 6: Logistic Regression training and cross-validation results.....	8
Table 7: Comparison of model characteristics.....	9
Table 8: Best Logistic Regression hyperparameters from GridSearchCV	9
Table 9: Best RBF SVM hyperparameters from GridSearchCV	10
Table 10: Final model comparison after hyperparameter tuning	11
Table 11: Digit 7 test data shapes	12

Task 1: Data Exploration and Analysis

1.1 Data Inspection

MNIST is actually generated by re-mixing from two NIST Special Databases, namely NIST Special Database 1 and 3. It is among the most popular benchmarks used in the machine learning research (LeCun et al., 1998).

This assignment uses a binary subset of MNIST handwritten digit database. It has only sample data for digits 1 and 7. It stores and has four arrays:

Dataset Component	Shape	Description
Training Images	(100, 28, 28)	100 grayscale images, 28x28 pixels
Training Labels	(100)	Integer labels: 1 or 7
Test Images	(1000, 28, 28)	1000 grayscale images, 28x28 pixels
Test Labels	(1000)	Integer labels: 1 or 7

Table 1: Dataset component shapes

1.2 Class Distribution

A training set is made up of 50 samples of digit 1 and 50 samples of digit 7. It results in a perfectly balanced binary classification problem. The following reasons are important:

- A balanced dataset means no class is neglected, reducing the risk of biased models.
- No oversampling or class weighting is required as a preprocessing step.

Digit	Number of Training Samples
1	50
7	50
Total	100

Table 2: Class distribution in training set

1.3 Pixel Statistics

To understand the pixel-level characteristics of each digit class, three statistics were computed across all training images:

Digit	Avg Pixel Intensity	Pixel Variance	Sparsity Ratio
1	20.18	4064.65	0.89
7	29.53	5606.38	0.83

Table 3: Pixel statistics per digit class

1.4 Average Digit Images

The average image for each class was calculated by taking the mean pixel value of all training samples of that class. This shows a sample shape of each digit as learned from the data.

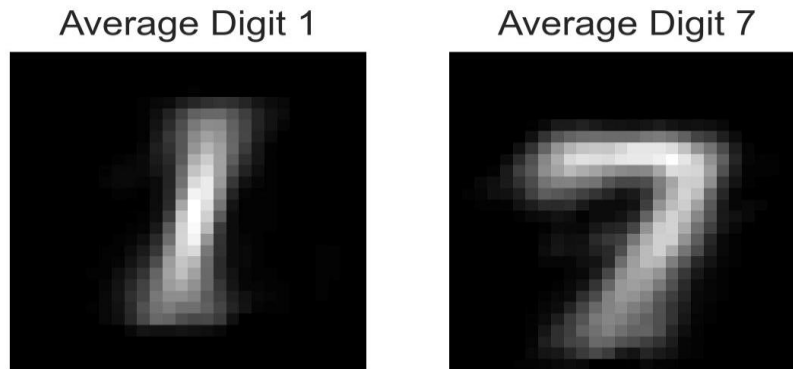


Figure 1: Average pixel images for digit 1 (left) and digit 7 (right)

The average images confirm the structural differences between the two digits. Digit 1 appears as a narrow stroke concentrated in the center of the image. Digit 7 displays a clear horizontal crossbar at the top and a diagonal downstroke.

1.5 PCA Projection

The scaled training data was transformed from a 784-dimensional feature space into 2D via Principal Component Analysis (PCA). PCA finds orthogonal directions of maximum variance in the data. It is useful for visualizing high dimensional feature spaces efficiently (Jolliffe and Cadima, 2016).

PCA Projection of Handwritten Digits 1 and 7

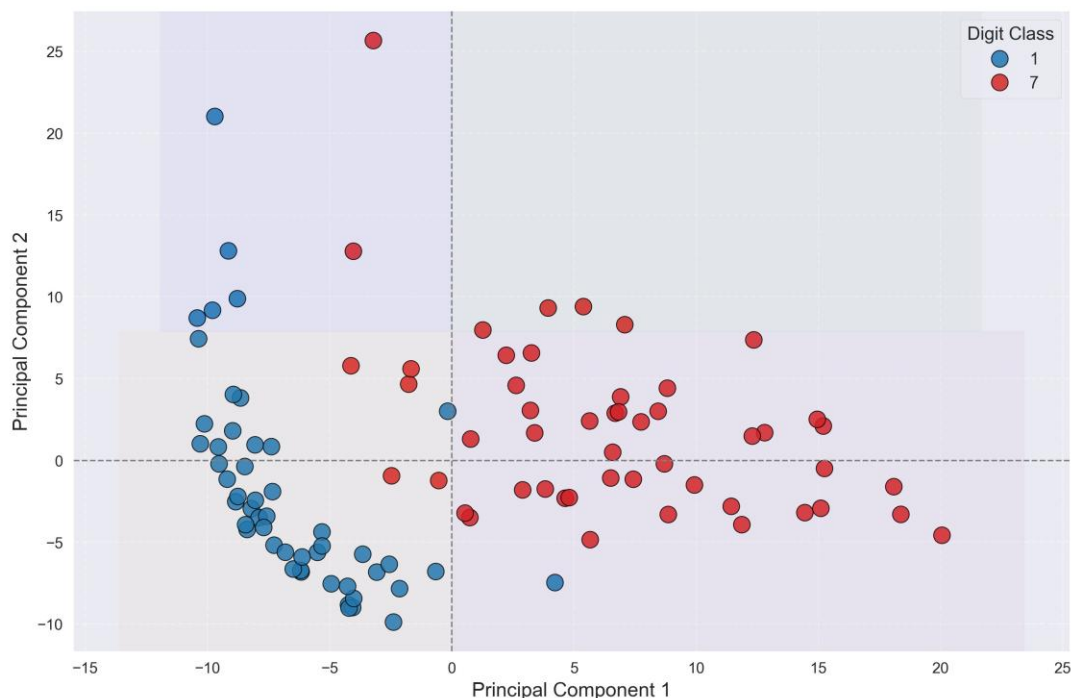


Figure 2: PCA projection of training data coloured by digit class

The PCA scatterplot shows a strong degree of linear separation between digit 1 (blue) and digit 7 (red). This suggests that a linear classifier can perform well on this task.

1.6 Data Preprocessing

The following preprocessing pipeline was applied to prepare the data for modelling:

Step	Operation
1. Flattening	Reshape (N, 28, 28) $\rightarrow$ (N, 784)
2. Normalization	Divide pixel values by 255.0 $\rightarrow$ [0, 1]
3. Standardization	StandardScaler: zero mean, unit variance

Table 4: Preprocessing steps and justifications

Data leakage prevention: `fit_transform` was only applied to the training set and then `transform` was applied to the test set. This is because test set statistics are not included in the calculation of the scaling parameters. This way it maintains the integrity of all evaluations downstream (**Géron, 2022**). The scikit-learn library was employed to implement all the machine learning applications in this report (**Pedregosa et al., 2011**).

Task 2: Build a Classification System

2.1 Model Choice: Logistic Regression

The logistic regression has been selected as the major classification algorithm because it is a binary classification problem. Logistic regression is shown to be a good baseline classifier in image-based tasks, particularly when there is limited training data (**Karuppasamy et al., 2024**). The following data-informed considerations are used to make the decision:

- **Linear separability observed in PCA:** From the PCA projection (Figure 1.2) it can be seen that digits 1 and 7 are linearly separable in the principal component space.
- **Small training set (100 samples):** In such a small training sample (100), complex non-linear models may end up being overfitted to the training data. Logistic Regression is a low variance model with an inbuilt value of the regularization parameter (C) that can be used with low sample sizes.
- **High-dimensional input (784 features):** Despite the high dimensionality, MNIST images are sparse. Logistic Regression can handle sparse high-dimensional data efficiently, especially with L1 or L2 regularization.
- **Interpretability:** Logistic Regression gives a linear decision boundary (and weights that are interpretable), which can be analyzed to see which pixels are most discriminant.
- **Computational efficiency:** Training is quick with the liblinear solver so it is feasible to do iterative hyperparameter tuning for cross-validation.

2.2 Evaluation Metrics

Multiple metrics were selected to provide a comprehensive evaluation of classifier performance:

Metric	Definition	Justification for Use
Accuracy	Proportion of all correctly classified samples	Valid because the dataset is perfectly balanced (50/50)
Precision	$TP / (TP + FP)$ for the positive class (digit 7)	Measures how many predicted 7s were truly 7s
Recall	$TP / (TP + FN)$ for the positive class (digit 7)	Measures how many actual 7s were correctly identified
F1-Score	Harmonic mean of precision and recall	Provides a single balanced measure; used as the primary CV optimization metric
CV F1-Macro	Average F1 computed equally across both classes	Appropriate for model selection to ensure balanced performance across both digit classes

Table 5: Evaluation metrics and justifications

F1-macro is used as the primary scoring metric during cross-validation because it treats both classes equally.

2.3 Implementation & Training

To achieve repeatability, the Logistic Regression model was instantiated with `max_iter=1000` and `random_state=42`.

A 5-fold Stratified K-Fold cross validation was done in addition to measuring the performance of the training set. The ratio of 50/50 classes was maintained in each fold by using Stratified K-Fold instead of standard KFold, this is particularly important given the small size of the data set.

2.4 Results

Model	Train Acc	Train Prec	Train Recall	Train F1	Mean CV F1	CV Std Dev
Logistic Regression	1.000	1.000	1.000	1.000	1.000	0.000

Table 6: Logistic Regression training and cross-validation results

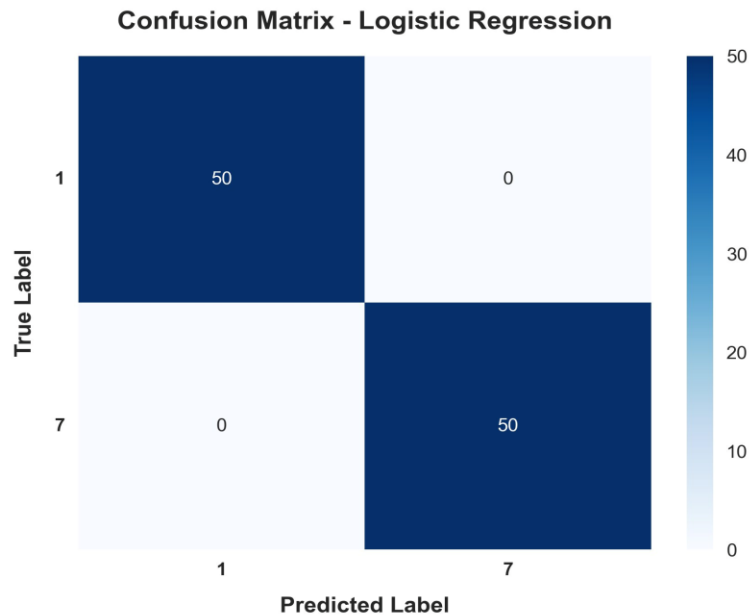


Figure 3: Confusion matrix for Logistic Regression on the training set

The Logistic Regression classifier performed well on the training set, with a score of 1 across all metrics, and no misclassifications for any of the two classes. The same trend was observed when implementing the 5-fold cross validation, which had a consistent mean F1-macro of 1.000 and a standard deviation of 0.000.

Perfect training performance may reflect overfitting, but the results of the cross validation on the five folds are very consistent, which strongly supports that the model is generalizing and not memorizing the training data. This result agrees with the high linear separability in the PCA projected version of this data set which makes Logistic Regression a suitable model choice for this data set.

Task 3: Comparison of Two Classifiers & Hyperparameter Tuning

3.1 Second Classifier: RBF Kernel SVM

A Support Vector Machine with a Radial Basis Function (RBF) kernel was selected as the second classifier. This choice deliberately contrasts with Logistic Regression in the following ways:

Dimension	Logistic Regression	RBF Kernel SVM
Decision boundary type	Linear	Non-linear
Feature space	Original 784-dim space	Implicitly infinite-dim via RBF kernel
Optimization objective	Minimizes log-loss (cross-entropy)	Maximizes margin between support vectors
Key hyperparameters	C (regularization), penalty type	C (regularization), gamma (kernel width)
Probabilistic output	Yes (direct probability estimates)	Not natively
Sensitivity to scale	Moderate	High

Table 7: Comparison of model characteristics

3.2 Cross-Validation Strategy

Throughout, a 5-fold Stratified K-Fold cross validation (Stratified K-Fold, `n_splits=5`, `shuffle=True`, `random_state=42`) was performed. For classification problems, Stratified K-Fold is recommended since it guarantees that each of the folds has the same class distribution as the original dataset (Vasan and Surendiran, 2024).

3.3 Logistic Regression Hyperparameter Tuning

Two hyperparameters were tuned for Logistic Regression:

- **C (regularization strength):** Values tested: {0.001, 0.01, 0.1, 1, 10, 100}. Lower C applies stronger regularization and higher C allows more flexibility.
- **Penalty type:** L1 (lasso) and L2 (ridge). L1 produces sparse weight vectors and L2 distributes weight more evenly. The liblinear solver was used as it supports both penalty types.

Key observations:

Metric	Value
Best C	0.1
Best Penalty	L2
Best Mean CV F1- Score	1.0

Table 8: Best Logistic Regression hyperparameters from GridSearchCV

3.4 RBF SVM Hyperparameter Tuning

Two hyperparameters were tuned for the RBF SVM:

- **C (regularization strength):** Values tested: {0.1, 1, 10, 100}. It controls the trade-off between maximizing the margin and minimizing training errors.
- **Gamma (kernel width):** Values tested: {0.001, 0.01, 0.1, 1}. It controls the influence radius of each support vector. Here, small gamma = broad influence and large gamma = narrow influence.

The heatmap below presents the full grid of mean CV F1-scores:

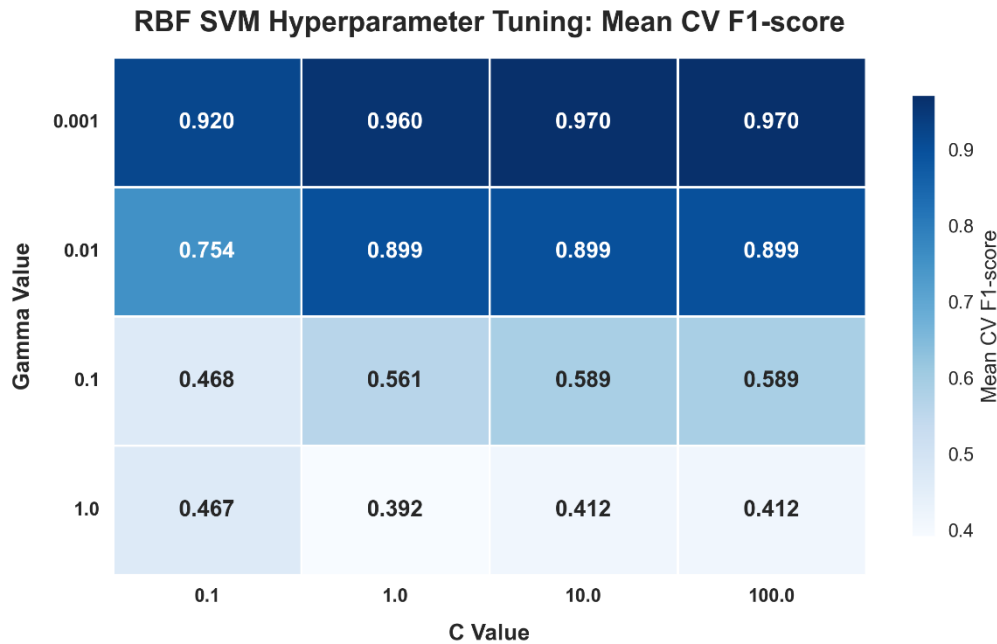


Figure 4: RBF SVM hyperparameter heatmap of mean CV F1-score (C vs gamma)

Key observations:

Best C	Best Gamma	Best Mean CV F1
10	0.001	0.970

Table 9: Best RBF SVM hyperparameters from GridSearchCV

3.5 Confusion Matrices on Training Data

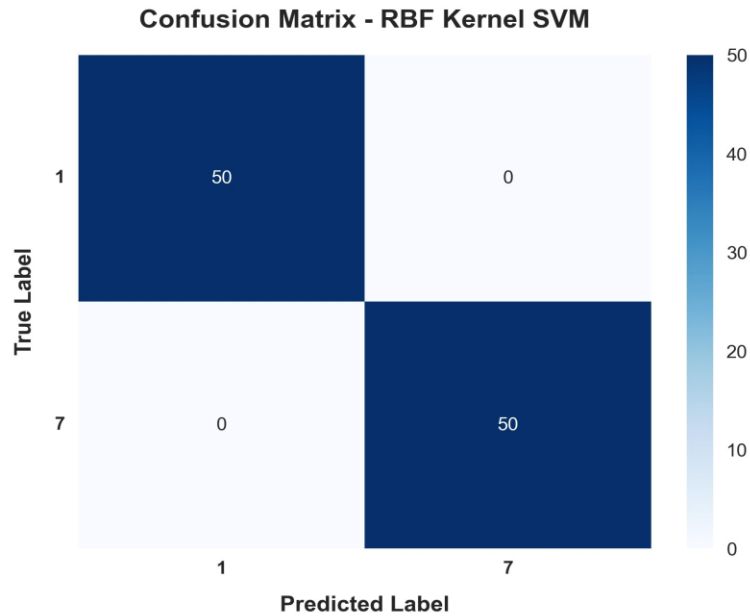


Figure 5: Confusion matrix for RBF Kernel SVM on the training set

Both classifiers achieve perfect classification on the training set (50/50 correct for each class, 0 misclassifications). This is consistent with both models as it has sufficient capacity to fit a well-separated binary problem with only 100 samples.

3.6 Final Model Comparison

Model	Best Parameters	Best Mean CV F1	Training F1
Logistic Regression	C=0.1, penalty=L2	1.000	1.000
RBF Kernel SVM	C=10, gamma=0.001	0.970	1.000

Table 10: Final model comparison after hyperparameter tuning

Discussion:

The Logistic Regression is better than the RBF SVM in the cross-validation (1.000 vs 0.970). It means a linear model is sufficient and it is probably more suitable for this dataset. The RBF SVM overfits the training set and gets a lower score in cross validation. The non-linearity of the kernel adds extra model complexity which is not required for a nearly linearly separable problem.

The major difference between these models is the type of their decision boundary. Logistic Regression learns a flat hyperplane in the 784-dimensional space whereas the RBF SVM learns an implicit higher dimensional space and a margin-maximizing boundary in this space. In this specific binary digit problem this additional complexity of the RBF mapping makes generalization little bit difficult.

Therefore, Logistic Regression with C=0.1 and L2 penalty is the best model for this task after cross validation because it is simple, easy to interpret and produces the best model.

Task 4: Clustering Analysis of the Test Set

4.1 Preparing the Digit 7 Test Data

For the clustering analysis, only digit 7 images from the test set were used. This subset was prepared as follows:

- Selected all test images where test labels == 7.
- Reshaped each 28x28 image into a flat 784-dimensional vector.
- Normalized pixel values to the [0, 1] range

Data Component	Shape
Digit 7 Test Images	(500, 28, 28)
Digit 7 Feature Vectors (normalized)	(500, 784)

Table 11: Digit 7 test data shapes

4.2 K-Means with 100 Clusters

As required by the task, K-Means clustering was first applied with $k=100$ clusters. K-Means is an unsupervised algorithm. It partitions data into k clusters by minimising within-cluster sum of squared distances to each cluster centroid (Govender and Sivakumar, 2020). This served as the baseline clustering. Here,

- `n_clusters=100`
- `random_state=42` for reproducibility
- `n_init=10` to run the algorithm with 10 different centroid seeds and retain the best result

4.3 Finding the Optimal Number of Clusters

Two standard methods were applied in combination to identify the optimal number of clusters:

4.3.1 Elbow Method (Inertia)

The inertia (sum of squared distances from each cluster to the cluster centroid) was calculated for 2 to 99 clusters. Tighter clusters means a lower inertia, but this is always a decrease as k increases. The elbow is the number of clusters at which the rate of decrease sharply decreases, the most efficient number of clusters. (Govender and Sivakumar, 2020).

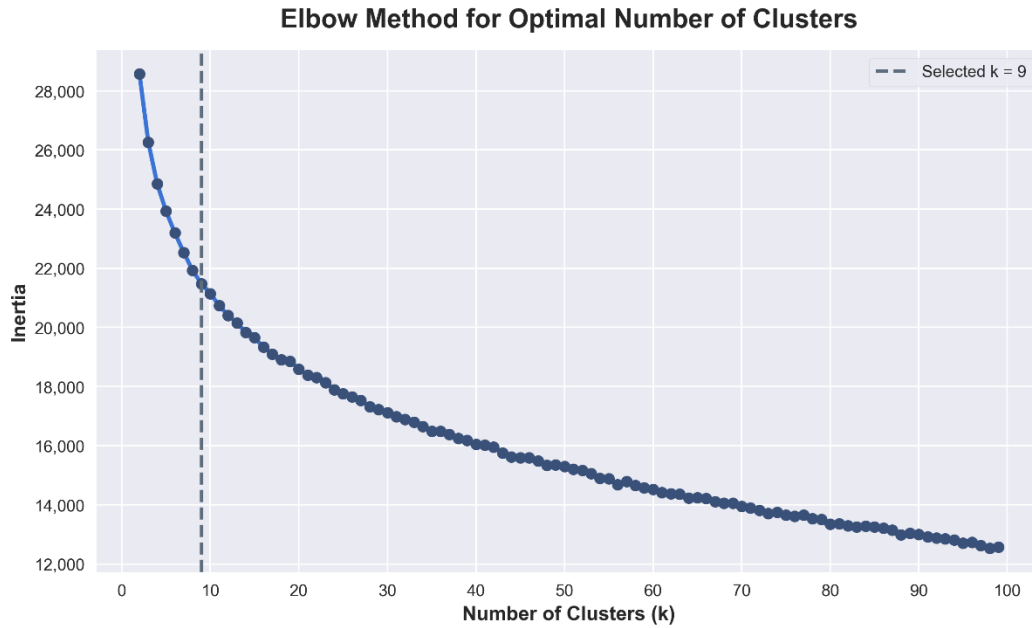


Figure 6: Elbow method: Inertia vs number of clusters ($k=2$ to 99)

The elbow plot indicates strong reduction of inertia between $k=2$ and approximately $k=9$ and then a decrease in the inertia curve. This may indicate that if $k>9$, the additional clusters will bring only a small reduction in within-cluster variance.

4.3.2 Silhouette Score

The Silhouette Score is a measure of the similarity of the point to its own cluster as compared to other clusters. The larger the silhouette score, the more clearly separated and defined the clusters are.

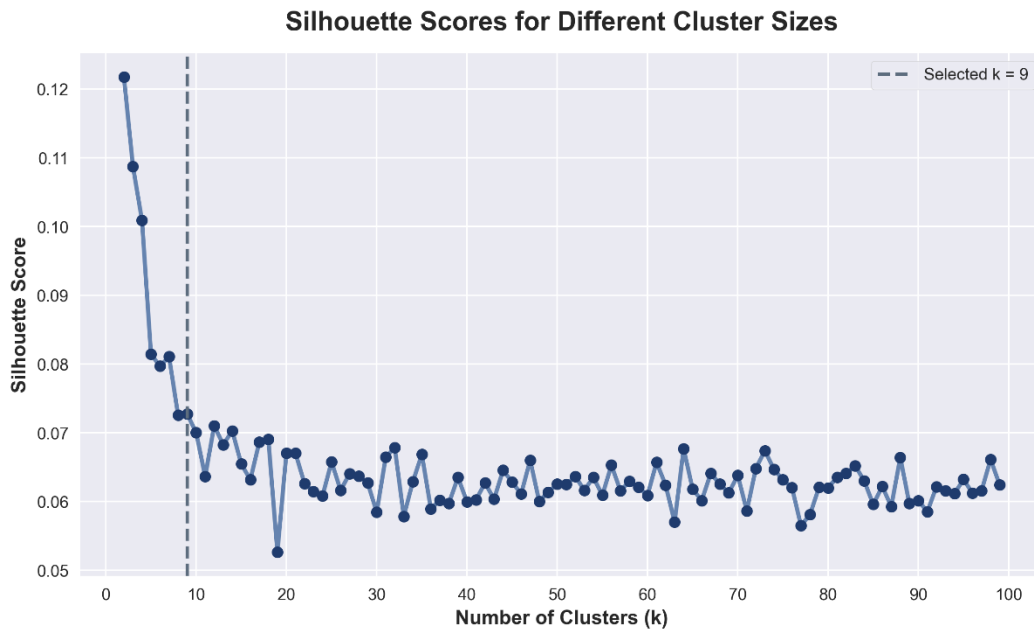


Figure 7: Silhouette scores for $k=2$ to 99

The silhouette score attains its maximum value at $k=2$ (around 0.122) and decreases with the increase of k . It converges to about 0.06 for $k>20$. $k=2$ gives the best silhouette score, but this is too coarse to get the variability for handwritten 7s. The score at $k=9$, there is a local highest which considerably match with elbow graph.

4.4 Final Cluster Selection: $k = 9$

Both the Elbow Method and Silhouette Analysis suggested that **$k=9$ is the best number of clusters**. Here is why:

Elbow Method: At $k=9$, the inertia starts dropping dropping quickly. This means adding more clusters less than 9 does not improve things much.

Silhouette Score: At $k=9$, the silhouette score is at a reasonable and stable level. This means the clusters are fairly well separated.

Interpretability: 9 clusters is a considerable number. Each cluster can represent a different handwriting style. It is detailed enough without being too complicated.

Practical Reason: The task requires k to be less than 100. $k=9$ fits within this limit and still gives meaningful and distinct clusters.

4.5.2 Cluster Centers

Each cluster center is the mean of all images. It means represents the "average" appearance of digit 7 within that cluster:

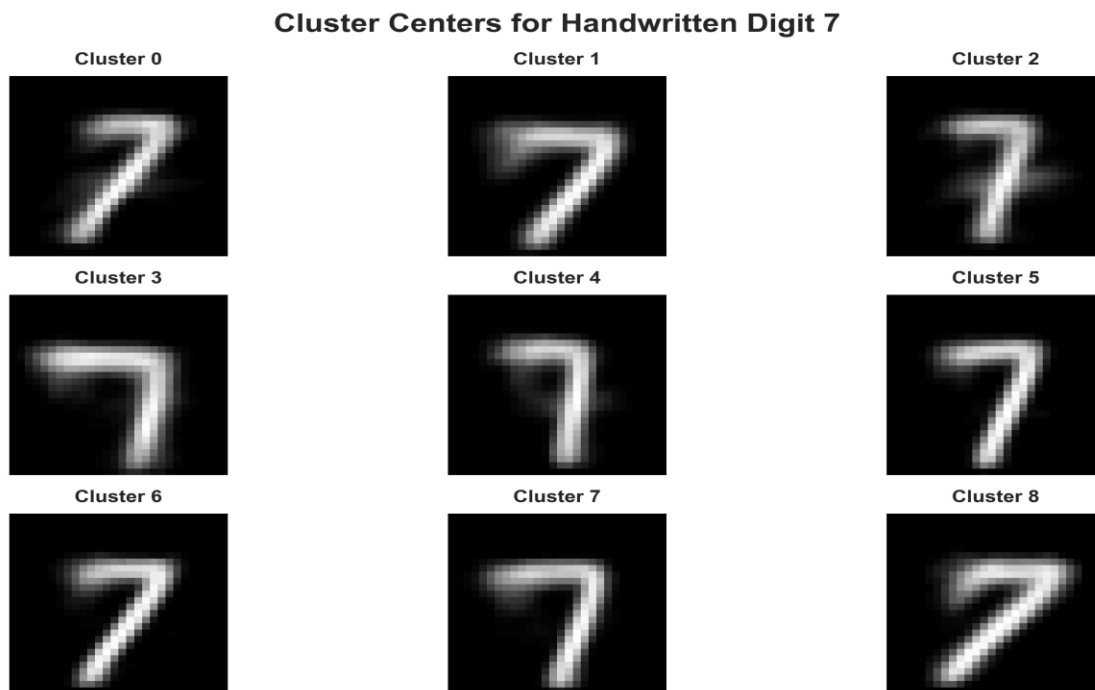


Figure 8: K-Means cluster centers ($k=9$) for digit 7 test images

The cluster centers show different ways people write the digit 7:

- **Clusters 0, 1, 2 and 5:** Clear and normal-looking 7's, straight top line and diagonal stroke. These are the most typical handwriting.
- **Cluster 3:** An enlarged larger 7 which shifted slightly to the left. This indicates that some people use larger strokes to write their number 7.
- **Cluster 4:** A smaller and more angular 7 with a more solid vertical line. It could be similar to the "1 with a small top line".
- **Clusters 6 and 7:** Slightly tilted 7s with difference in writing angle.
- **Cluster 8:** A smaller 7 but lower and to the right in the image.

4.5.3 Cluster Size Distribution

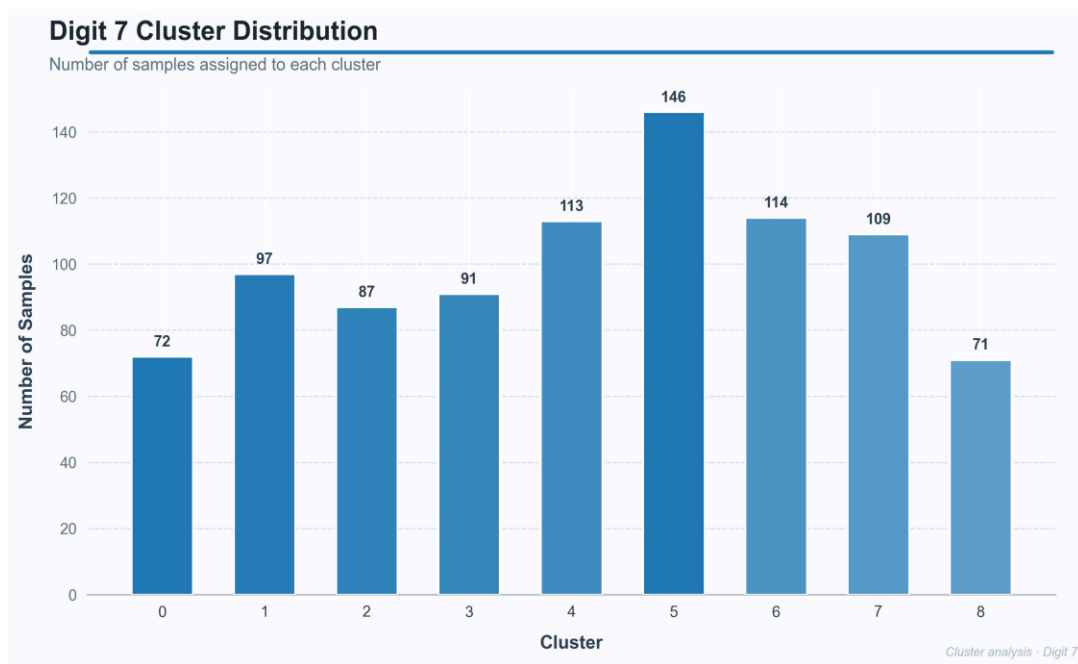


Figure 9: Distribution of samples per cluster for digit 7 ($k=9$)

The most common way people write the digit 7 in the test set is in cluster 5 with 146 images (18.5%). On the other hand, clusters 0 and 8 are the smallest in number (71-72 images, about 9% each). It implies that these are less common but distinct handwriting styles. In general, size distribution of the clusters is reasonably balanced with all the digit 7 written in various styles. This means that the K-Means algorithm has been able to discover meaningful clusters of handwriting patterns from the dataset.

4.6 Summary

The clustering analysis was able to successfully cluster the different handwriting styles of the digit 7. Elbow Method and Silhouette Score were used to determine the optimal number of clusters which found $k = 9$. The cluster centers showed clear differences in how people write the digit 7. It simplifies the interpretation of the clusters. The sizes of the clusters were also relatively even. It means that there are multiple handwriting styles in the data set. In a nutshell, K-Means effectively grouped similar handwritten digits into meaningful clusters.

References

- Byun, H., Kim, M., Park, J. and Lee, S. (2024)** Research on image classification of RBF kernel SVM. *IEEE International Conference on Intelligent Computing and Signal Processing*. IEEE. doi:10.1109/ICSP60620.2024.10948583.
- Géron, A. (2022)** Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems. 3rd edn. Sebastopol, CA: O'Reilly Media.
- Govender, P. and Sivakumar, V. (2020)** Application of k-means and hierarchical clustering techniques for analysis of air pollution: A review (1980–2019). *Atmospheric Pollution Research*, 11(1), pp. 40–56. doi:10.1016/j.apr.2019.09.009.
- Jolliffe, I.T. and Cadima, J. (2016)** Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065), p. 20150202. doi:10.1098/rsta.2015.0202.
- Karuppasamy, A., Abdesselam, A., Hedjam, R., Zidoum, H. and Al-Bahri, M. (2024)** Feed-forward networks using logistic regression and support vector machine for whole-slide breast cancer histopathology image classification. *Intelligence-Based Medicine*, 9, p. 100126. doi:10.1016/j.ibmed.2023.100126.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998)** Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), pp. 2278–2324. doi:10.1109/5.726791.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011)** Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html> (Accessed: 20 May 2026).
- Vasan, D. and Surendiran, B. (2024)** Comparative analysis of cross-validation techniques: LOOCV, k-folds cross-validation, and repeated k-folds cross-validation in machine learning models. *American Journal of Theoretical and Applied Statistics*, 13(5), pp. 127–137. doi:10.11648/j.ajtas.20241305.13.